Vihaan Manchanda

Duke University · Explainable AI

prompting-explained

Which Words in Your Prompt Actually Matter?

A teaching tool that shows you the impact of each phrase in your prompt on the AI's answer.

SCAN TO TRY THE APP

github.com/Vihaan8/prompting-explained

01 Executive summary

People in almost every field now use AI tools, but there is no tool that teaches prompt-writing literacy. AI developers learn what works through practice. Most users never do. In one study, non-experts wrote prompts the way they would instruct a person, and struggled to fix them when the answer went wrong (Zamfirescu-Pereira et al., 2023). Small wording changes matter too. Changing only a prompt's formatting moved one model's accuracy by up to 76 points (Sclar et al., 2024). A badly worded prompt wastes tokens and context, and gets a worse answer.

I built **prompting-explained** to teach how each word in a prompt contributed to the AI's answer. It removes one phrase at a time, reruns the prompt, and highlights the phrases that changed the result. On 240 test prompts across 8 everyday tasks, the phrases it ranked highest mattered clearly more than random ones, and its top pick repeated 78% of the time.

The goal is a training tool companies load with their own tasks, so staff learn what works on their real work.

02 The problem

AI tools show you an answer, but not which of your words produced it. So people keep habits that don't help, and some mistakes slip through. In my tests, asking the AI to turn a support ticket into a public bug report leaked the customer's name in 2 of 3 runs, with no warning.

03 Who it's for

Anyone learning to prompt. Practice on everyday tasks and see how the AI read your words.

Companies. Add your own tasks, documents and rules, so staff learn on the work they actually do.

Governance teams. Test whether a safety instruction works before rolling it out.

RESEARCH QUESTION

Which parts of an everyday prompt actually change an AI's answer, and can removing parts and rerunning explain that **faithfully** and **stably**?

04 Methods

How the tool works out what each phrase did, in five steps. Starred terms are explained in the purple notes.

1 Split the prompt into phrases

An AI splits the prompt into up to 10 phrases and tags each one (task, audience, format, politeness...). These phrases are the pieces the tool will perturb*. Code keeps each phrase exactly as typed, typos included.

*** Perturbation:** changing part of an input to see how the output responds. LIME does this with single words.

2 Remove one phrase and rerun

The full prompt runs 3 times. Then each phrase is deleted in turn and the shorter prompt runs twice. This is occlusion*. A phrase's effect is how much the score drops without it, which gives every phrase its own counterfactual**. Phrases that change nothing alone are also removed together once, in case they cover for each other.

*** Occlusion (leave-one-out):** hide one part of the input, rerun, and measure the change.
**** Counterfactual:** the same case with one thing changed, to see what that one thing did.

3 Score every answer two ways

Against the task. Each task comes with a short checklist that code grades automatically, like a teacher's rubric, from 0 (misses everything) to 1 (passes everything). An answer that doesn't do the task at all, such as asking "what should I explain?", scores 0.

Against your instructions. Code can't check everything you might ask for, such as "use a candy example". So a second AI reads each answer and ticks off which of your instructions it followed*. That separates "the AI followed this" from "this made the answer better".

*** AI as a judge:** using one AI to grade another's output. It can be wrong, so its mistakes are reported.

4 Filter out noise

The same prompt gives a slightly different answer each time, so every version runs more than once. A change only counts if it's bigger than the noise floor*. Anything smaller stays grey, and the tool says "not clear" rather than guessing.

*** Noise floor:** the smallest change the tool can tell apart from the AI's natural randomness (two standard errors).

5 Explain it in plain words

Each phrase that mattered gets one sentence, written by an AI, describing how the answer changed without it. Every number and every color comes from code.

05 Worked example

Task: write a prompt that gets the AI to explain inflation to your 10-year-old cousin. One user wrote this:

> Hi there! Explain inflation to my 10-year-old cousin. Use exactly 3 bullet points. Mention a candy bar in one of them. Use proper technical terms.

Made the answer better · Followed, no change in score · Worked against the task · No real difference

Green · "Explain inflation": without it, the AI explained how the body uses food for energy.

Blue · "Mention a candy bar": the AI did it, but the score was the same either way.

Red · "Use proper technical terms": the AI used words like "Consumer Price Index", too hard for a 10-year-old. Without it, the score rose from 0.40 to 0.78.

Grey · "Hi there!": a greeting gives the AI nothing to act on.

Scores run 0 to 1. On a fresh rerun, 5 of 6 verdicts repeated. The "without it" sentences are written by an AI and labelled as descriptions; every number is computed by code.

06 Can the tool be trusted?

A teaching tool that points at the wrong phrases would teach the wrong habits. So I ran it on 240 practice prompts, written at beginner, average and expert level for all 8 tasks, plus 32 deliberately messy ones.

What held up

Removing the two phrases it ranked highest dropped the score by **0.30**, against **0.18** for two random phrases (79 prompts, p = 0.0002). So it points at phrases that really matter, which makes its explanations faithful*.

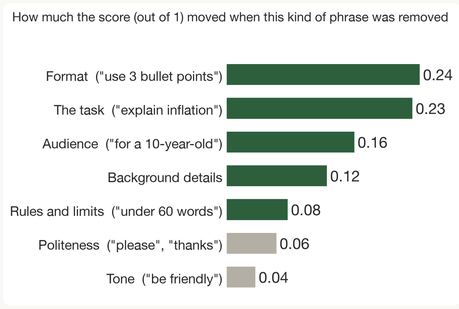
Running the whole analysis again picked the same top phrase **78%** of the time, and 86% when the effect was clear, so the explanations are stable**.

*** Faithfulness:** the parts an explanation points to really drive the output.
**** Stability:** the same input gets the same explanation when run again.

Some results surprised me. "dont guess" made the AI speculate *more*, in all 5 runs. Asked to "make up a salary", it refused. "for github issues" made it leave a customer's name out of a bug report without being asked.

Which kinds of phrases mattered in these tests

Format instructions and the task itself moved the score most. Politeness and tone barely moved it.



07 Where it falls short

- When an attached document already held the answer, as in the landlord and meeting tasks, removing two phrases moved the score by less than 0.07. There was little to rank.
- The AI that checks instructions can't count words. It agreed with code 88% of the time, but passed 93 answers that broke a word limit.
- Running each prompt more times barely helped. Nearly tripling the calls raised agreement from 83% to 91%.
- My hallucination test never triggered. The model didn't invent a salary, even with a bonus in the posting as bait.

08 Business implications

- Companies can load their own tasks, documents and rules, so employees practice on real work and see which habits change results.
- People outside AI teams can see how the model read their words, with no technical background.
- As a stretch goal, once enough phrases are measured, results can be cached and a small local model trained. An app or browser extension could then check a prompt before it is sent, without an extra AI call, and cut token costs.

09 Limitations and next steps

- One model (Claude Haiku 5.5), on tasks with checks written in code.
- The 240 test prompts were written by an AI imitating users; I wrote the 32 messy ones. There is no study with real users yet.
- Each phrase is compared only with deleting it, not with a better phrase.
- Phrases that do nothing alone are retested as one group, but not in every combination. Shapley values* would, at a much higher cost.
- The splitter and instruction checker are AIs and can be wrong. Checked against code, the checker agreed 88% of the time but can't count words.
- **Next:** Shapley-style scoring, a study with real learners, more models.

*** Shapley values:** give each part a fair share of the credit by testing it in every combination of parts.

REFERENCES Zamfirescu-Pereira et al. (2023), Why Johnny Can't Prompt, CHI · Sclar et al. (2024), prompt formatting sensitivity, ICLR · Zeiler & Fergus (2014), occlusion · Ribeiro et al. (2016), LIME · Lundberg & Lee (2017), SHAP · Petsiuk et al. (2018), deletion test · DeYoung et al. (2020), faithfulness · Alvarez-Melis & Jaakkola (2018), stability · Turpin et al. (2023), unfaithful model explanations